

Article

A Recommender System Model for Presentation Advisor Application Based on Multi-Tower Neural Network and Utility-Based Scoring

Maria Vlahova-Takova and Milena Lazarova * 

Faculty of Computer Systems and Technologies, Technical University of Sofia, 1000 Sofia, Bulgaria; mariavlahova@tu-sofia.bg

* Correspondence: milaz@tu-sofia.bg

Abstract

Delivering compelling presentations is a critical skill across academic, professional, and public domains—yet many presenters struggle with structuring content, maintaining visual consistency, and engaging their audience effectively. Existing tools offer isolated support for design or delivery but fail to promote long-term skill development. This paper presents a novel intelligent application, the Presentation Advisor application, powered by a personalized recommendation engine that goes beyond fixing slide content and visualization, enabling users to build presentation competence. The recommendation engine leverages a model based on hybrid multi-tower neural network architecture enhanced with temporal encoding, problem sequence modeling, and utility-based scoring to deliver adaptive context-aware feedback. Unlike current tools, the presented system analyzes user-submitted presentations to detect common issues and delivers curated educational content tailored to user preferences, presentation types, and audiences. The system also incorporates strategic cold-start mitigation, ensuring high-quality recommendations even for new users or unseen content. Comprehensive experimental evaluations demonstrate that the suggested model significantly outperforms content-based filtering, collaborative filtering, autoencoders, and reinforcement learning approaches across both accuracy and personalization metrics. By combining cutting-edge recommendation techniques with a pedagogical framework, the Presentation Advisor application enables users not only to improve individual presentations but to become consistently better presenters over time.

Keywords: recommender system; presentation improvement; personalization; deep learning; multi-tower neural network; utility-based scoring; deep Q-network



Received: 2 June 2025
Revised: 19 June 2025
Accepted: 20 June 2025
Published: 22 June 2025

Citation: Vlahova-Takova, M.; Lazarova, M. A Recommender System Model for Presentation Advisor Application Based on Multi-Tower Neural Network and Utility-Based Scoring. *Electronics* **2025**, *14*, 2528. <https://doi.org/10.3390/electronics14132528>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Presenters often struggle with maintaining consistency, engaging the audience, and structuring content effectively. Existing tools provide optimization features but often fall short in teaching core design principles for designing flawless presentations. Instead of providing a tool that enables fixing a presentation, the Presentation Advisor application presented in this paper is designed to be a learning tool that supports building users' skillsets. The goal of the Presentation Advisor is to help users improve their presentations by identifying potential problems—such as an unclear structure, weak arguments, inappropriate visuals, or low readability—and offering tailored suggestions and supporting literature. These tailored suggestions are personalized recommendations for how to revise or enhance

specific aspects of the presentation, considering the user's goals (e.g., academic, business, or educational), audience type, and topic. The supporting literature may include academic papers, best practice guides, or curated learning resources relevant to the identified issues, helping users not only correct problems but also understand the reasoning behind the suggested improvements.

At the core of the system lies a recommender engine designed to evolve for each user over time, thereby fostering long-term learning and engagement. The cold-start problem is strategically mitigated in the system architecture, ensuring the effective handling of new users and items through adaptive modeling techniques.

The paper presents the architecture, methodology, and evaluation of the proposed recommendation engine.

The main contributions of this work are as follows:

- A personalized recommendation system is designed and implemented that supports skill building in presentation creation through contextualized feedback and resources;
- A hybrid recommendation model is proposed that combines deep learning and utility-based scoring to enhance the relevance and adaptability of the suggestions;
- The cold-start challenge is addressed by leveraging feature-based modeling and problem sequence learning to improve recommendations for new users and content.

The rest of the paper is organized as follows: Section 2 reviews related works in the field of presentation tools and recommender systems. Section 3 outlines the data generation and analysis process. Section 4 details the architecture and design of the recommendation model. Section 5 describes the methodology used, including feature engineering and model training. Section 6 discusses the evaluation of the experimental results, challenges encountered, and proposed solutions. Finally, Section 7 summarizes the paper's contributions and outlines directions for future research.

2. Related Work

2.1. Presentation Advisor Tools

Two of the most popular tools—"Slidewise" and "PitchVantage"—that can assist the improvement of users' presentations are discussed in this section. Table 1 shows a comparison of "Slidewise" and "PitchVantage" based on their main features and limitations, as well as an evaluation of the tools as competitors or potential partners of the Presentation Advisor.

Table 1. Comparison of existing presentation tools.

Tool	Features	Limitations	Competitor
"Slidewise" [1]	Identifies and corrects font mismatches, image sizes, and layout discrepancies	No educational feedback; Only paid version is available	Yes
"PitchVantage" [2]	Presentation delivery coaching with AI	Does not support PPT slides	Not a competitor but an integration candidate

"Slidewise" [1] is a specialized tool designed to enhance the management and optimization of Microsoft PowerPoint presentations. "Slidewise" can be used with Microsoft PowerPoint 2013, 2016, 2019, 2021, and Microsoft 365 (Windows versions). It helps users ensure consistency in their slides by identifying and correcting font mismatches, image sizes, and layout discrepancies. The software automatically detects missing fonts and

allows for fonts to be easily replaced across the entire presentation. It also offers image optimization features, reducing file sizes without sacrificing visual quality. “Slidewise” is particularly valuable for teams who need to maintain brand consistency across multiple presentations. The tool provides a clear overview of all the elements used in a presentation, making it easier to identify and address issues. By automating many time-consuming tasks, “Slidewise” saves considerable effort in preparing polished, professional presentations and supports professionals looking to streamline the presentation creation process while ensuring high standards of consistency and quality. It integrates directly with Microsoft PowerPoint, making it accessible and easy to use within existing workflows.

The main advantages of “Slidewise” are as follows:

- Font and image management helps ensure that the fonts are consistent and images are optimized;
- Slide consistency: checks for uniformity in design elements like colors, fonts, and layouts;
- Presentation optimization identifies and automatically resolves technical issues within the presentation, such as missing fonts or large file sizes.

The main disadvantages of “Slidewise” are pointed out as follows:

- Does not offer educational content or recommendations on design principles, although it excels in technical presentation management;
- Does not measure whether the content in a presentation is eye-catching;
- Price: the free version provides only a few essential features; the paid version is licensed as part of the user’s annual subscription.

“PitchVantage” [2] is an AI-powered presentation training tool designed to help users improve their public speaking and delivery skills. It focuses on the performance aspect of presentations rather than the slide content itself. Users practice speaking while the tool gives real-time feedback on various elements, such as filler words, pacing, tone, volume, and eye contact. Thus, the tool encourages users to become more confident and engaging speakers. “PitchVantage” creates a realistic environment by simulating an audience, which allows users to rehearse under conditions like live presentations. The tool is especially useful for students, educators, and professionals who wish to develop their communication skills. However, “PitchVantage” does not support or analyze PowerPoint slides directly, which limits its use in terms of content improvement or slide design.

The main advantages of “PitchVantage” are as follows:

- AI coaching for public speaking provides real-time feedback on delivery, such as tone, pacing, and filler words;
- A simulated practice environment offers a virtual audience to make practice sessions feel more realistic;
- Skill improvement: helps users build confidence and improve their presentation delivery over time.

The main disadvantages of “PitchVantage” include the following:

- The lack of slide integration: does not interact with or analyze PowerPoint slides, making it less useful for content or design feedback;
- Content evaluation: does not assess whether the slides or spoken messages are engaging or well structured;
- Price: the pricing may vary and it is typically offered as part of educational or organizational packages.

2.2. Recommender Systems

Recommender systems (RSs) [3–9] are designed to predict the utility of items for users to facilitate personalized recommendations. These predictions, often based on a real-valued function $R(u, i)$, estimate the utility of an item ‘i’ for a user ‘u’. The utility is used to recommend items with the highest predicted value. Various types of RSs include the following:

- Content-Based RSs: suggest items like those the user has previously liked, based on the attributes or features of the items [10];
- Collaborative Filtering RSs: recommend items based on the preferences of users with similar tastes, utilizing user rating histories to find patterns [11,12];
- Knowledge-Based RSs: recommend items based on specific domain knowledge about how item features meet user needs [13–16];
- Hybrid Systems RSs: combine multiple recommendation techniques to leverage the strengths of each and mitigate their weaknesses [17–19];
- Community-Based RSs: recommend items based on the preferences of a user’s social circle, assuming people within a network share similar tastes [20,21];
- Context-Aware RSs: take into account contextual information like location or time, which might affect the relevance of recommendations [22,23];
- Utility-Based RSs: estimate the usefulness of a product or service to a particular user based on a utility function [24,25]. The utility function is often unique for each user and can be influenced by the various attributes of the items being recommended. The utility is a measure of satisfaction or benefit that a user receives from consuming an item, and it can be derived from explicit preferences, such as ratings, or inferred from implicit signals, such as their browsing history or purchase patterns.

In recent years, deep learning has emerged as a powerful paradigm in the field of recommender systems, offering substantial improvements over traditional collaborative filtering and content-based methods. By leveraging multi-layer neural architectures, deep learning models can effectively capture complex non-linear user–item interactions and high-dimensional feature representations [26–28].

Each RS type addresses different scenarios, and the user needs to enhance the overall effectiveness of recommendations through a combination of approaches. Some RSs might not fully estimate utility before making recommendations but use heuristics based on various knowledge types about users and items, thereby optimizing the recommendation process.

2.3. Popular Recommender Systems

2.3.1. Netflix Recommender System

Netflix employs a multi-algorithm recommendation system that integrates collaborative filtering, deep learning, and reinforcement learning to deliver highly personalized content [29]. The key approaches utilized in the Netflix recommender systems can be summarized as follows [30–32]:

- Matrix factorization extracts latent user–item preferences from interaction data;
- Personalized ranking: focuses on ranking content based on user preferences rather than predicting ratings;
- Deep learning models use convolutional neural networks (CNNs) for visual analysis, recurrent neural networks (RNNs) and transformers for sequential patterns, and autoencoders for cold-start scenarios;
- The hybrid approach combines collaborative filtering with content-based methods to improve recommendation accuracy;

- Contextual bandits and graph neural networks (GNNs): adapt recommendations dynamically based on real-time user behavior. When a user visits Netflix, the system must select and rank content to display. Instead of relying solely on historical data, contextual bandits explore and exploit by testing different recommendations and learning from user feedback (e.g., clicks and watch duration). This allows recommendations to adapt dynamically, considering factors like time of day, device type, and recent viewing behavior;
- Diversity and serendipity models ensure that users receive a mix of relevant and unexpected content to enhance engagement;
- Multi-armed bandits for cold-start problems: to address the case in which there is not much information about a new user or item. This approach carefully balances exploring new options and using what is already known to give good recommendations. This balance helps the system learn user preferences step by step, even with very few data at the beginning.

Netflix continues to refine its recommendation strategies by leveraging context-aware models, embedding techniques, and attention mechanisms, improving personalization while addressing scalability and interpretability challenges.

2.3.2. Amazon Recommender System

Amazon's recommendation engine [33–35] is a key driver of its eCommerce success, enhancing customer satisfaction and sales through personalized shopping experiences. In 2003, Amazon introduced item-to-item collaborative filtering, a scalable and computationally efficient alternative to traditional user-based filtering. This approach analyzes a user's purchase history to suggest related items based on purchase likelihood, improving real-time recommendation accuracy while addressing biases like frequent buyer tendencies. Over time, Amazon expanded its recommendation system to include neural networks, particularly for Prime Video, in which autoencoders predict short-term user preferences using temporal data. By integrating user-based filtering, item-based filtering, and content-based approaches, Amazon significantly improved its recommendation quality. More recently, Amazon has incorporated self-attentive sequential recommendation models and LLMs (large language models) to further refine its recommendations and address the cold-start problem. In addition, to enhance diversity and relevance, Amazon employs clustering-based sampling (which groups items based on metadata for diverse selections), determinantal point processes (which reduce redundancy in recommendations), and category-aware filtering (which ensures coverage across multiple genres or subcategories). Other key approaches used are zero-shot recommendation systems and diversity maximization by semantic embedding models, prompt-based learning (LLMs) [36,37], transfer learning, and contrastive learning [38–40].

2.3.3. LinkedIn Learning Recommendation Engine

The LinkedIn learning recommendation engine employs an AI-driven recommendation system that utilizes deep learning techniques to personalize course suggestions [41,42]. The system is designed to optimize engagement and learning outcomes through neural collaborative filtering (Neural CF) and response prediction models. The general architecture of the LinkedIn learning recommendation engine comprises the following:

- Computes learner and course embeddings: generates embeddings for all courses based on past engagement;
- Ranks courses for each learner: computes ranking scores using the Neural CF output layer;

- Filters and finalizes recommendations: selects the top k courses for final recommendations.

The key approaches utilized in the LinkedIn learning recommendation engine are as follows:

- Neural collaborative filtering [43]: Neural CF consists of two multi-layer neural networks: the learner network and the course network. The learner network encodes a learner's past watched courses into a sparse vector representation, while the course network represents similarities between courses based on co-watching patterns. These networks feed into an embedding layer, which generates learner and course embeddings, effectively capturing relationships between users and content. Finally, the output layer computes a personalized recommendation score ranking courses for each learner based on their predicted relevance. The big advantage of this two-tower architecture is the separate training of the learner and the course embeddings, thus allowing for the reuse of tasks, such as finding related courses and supporting the model's explainability;
- Response prediction model: the response prediction component improves course recommendations by modeling learner–course interactions using a generalized linear mixture model (GLMix) [44]. The GLMix learns global model coefficients shared across all learners as well as the per-learner and per-course coefficients. The recommendations are customized based on individual learning patterns as well as recent improvements that integrate course watch time as a weight factor, ensuring that courses with higher engagement are prioritized.

2.3.4. YouTube Recommender System

YouTube's video recommender system is built on a deep-learning-based two-stage framework, comprising candidate generation and ranking networks [45]. The candidate generation network utilizes a neural-network-based collaborative filtering approach, by which user history and context are encoded into dense embeddings. These embeddings are learned using a "SoftMax" classifier that predicts the probability of a user watching a specific video. The embeddings for videos are pre-trained using continuous bag-of-words (CBOW) language models, ensuring the effective representation of video relationships based on co-watching patterns. The model incorporates various input features such as watch history, search behavior, and demographics, which are processed through fully connected layers with ReLU activations to generate a set of highly relevant candidate videos. The ranking network further refines these candidates by assigning a relevance score based on a richer set of features. It employs a weighted logistic regression, by which the model optimizes the expected watch time rather than simple clicks. The network uses both categorical and continuous features, normalizing numerical inputs like "time since last watch" and "previous impressions" while applying power transformations to enhance expressive capabilities. Similar to the candidate generation model, it utilizes deep feed-forward neural networks learning user–video interaction patterns to predict viewing likelihood.

To handle multi-objective optimization, YouTube leverages a multi-gate mixture of experts (MMoE) model [46]. The MMoE model consists of multiple expert neural networks, each specialized for different objectives such as engagement (clicks and watch time) and satisfaction (likes, shares, and ratings). By employing soft parameter sharing, the system dynamically selects which expert networks contribute to each objective, balancing short-term engagement with long-term user retention.

To address position bias, YouTube's recommender system incorporates a shallow tower network that models positional effects in rankings [47]. The network predicts position bias

logits, which are then combined with the ranking scores to ensure that video placement does not unfairly influence recommendations. The system is trained using implicit feedback learning, by which the watch time is used as a proxy for positive engagement rather than relying on explicit user ratings.

For training, YouTube's recommender system employs sequence-based learning techniques using predictive models to forecast future video engagements. The training process avoids information leakage by restricting input data to past watch history and search queries, ensuring that predictions reflect real-time user behavior. Additionally, the model benefits from domain-adaptive learning, in which embeddings are continuously updated to accommodate new content and shifting user interests.

Through a combination of neural collaborative filtering, deep feed-forward networks, multi-task learning with MMoE, and implicit feedback optimization, YouTube's recommender system ensures highly personalized and scalable content discovery, adapting dynamically to evolving user preferences.

3. Data Generation and Exploratory Analysis for Presentation Recommender System

3.1. Data Generation

To design and train a recommendation model for a presentation recommendation system, a custom-generated dataset is used that comprises all required data for users, articles, interactions, and ratings. The dataset is synthesized using a structured methodology through issue-based templates, random selection, temporal weighting, and user-defined mappings [48]. The goal is to generate a realistic dataset that enables effective personalized recommendations for improving presentations.

The dataset generation is guided by several predefined categories as follows:

- Common presentation challenges, such as lack of engagement, poor readability, or inconsistent design. These challenges are systematically listed and categorized in Table 2;
- Presentation types, which define the context or purpose of a presentation such as academic lectures, business pitches, or training sessions. A detailed classification is presented in Table 3;
- Audience types, encompassing different viewer profiles like executives, students, or the general public, each with distinct expectations and comprehension levels. A comprehensive overview of these audience categories is shown in Table 4.

The common presentation challenges are defined through a structured requirement-gathering phase based on established presentation design principles [49–52]. Specifically, ten categories are identified that reflect key factors affecting audience engagement and slide quality, such as readability, content layout, visual appeal and layout, the use of graphics, the presence of a structure (e.g., agenda slides or adherence to the 7×7 rule), and consistency. The key categories that are identified are translated into binary labels to simplify detection and classification. While the strategy of using the ten selected categories simplifies the complexity of real-world presentation issues, it serves as a practical starting point and allows for further deployment with real users and data, enabling the iterative expansion and refinement of the problem space based on real-world feedback and observations.

Table 2. Common presentation challenges.

Challenge	Brief Description
Boring presentation	Slides fail to engage the audience; lacking energy or visual interest
Graphics	Graphics are unclear, overly complex, or not suitable for the content
Readability	Text is hard to read due to the font choice, color contrast, or background
Consistency	Inconsistent styles across slides—the fonts, colors, or layouts vary too much
Images	Poor-quality, irrelevant, or excessive images
Bullets	Overuse of bullet points, making slides feel monotonous and cluttered
Text size	Text is either too small or too large, disrupting visual hierarchy and clarity
Text-heavy	Slides contain too much text, making them hard to scan or follow
Tables	Tables are overly detailed, dense, or hard to interpret quickly
Agenda	Lacks clear structure or navigation—audience may feel lost during the presentation
Infographics	Infographics are confusing, overly complex, or do not effectively support the message
Positioning	Elements on slides are misaligned or poorly arranged, affecting readability and flow

Table 3. Presentation types.

Presentation Type	Brief Description
Formal	Structured and professional; often used in corporate or official settings
Creative	Visually engaging and innovative; uses storytelling, design, and non-linear flow
Technical	Focused on detailed data, processes, or research; often for expert or specialized audiences
Business	Aimed at decision making, pitching, or reporting in a business context
Educational	Designed to teach or explain a topic; commonly used in classrooms or workshops
Persuasive	Intended to convince or influence the audience to move towards a viewpoint or action

Table 4. Audience types.

Audience Type	Brief Description
Academic	Students, researchers, or educators in educational institutions
Business	Professionals in corporate environments; decision makers, managers, or teams
Technical	Engineers, developers, analysts, or any audience with specialized technical expertise
Kids	Young children or school-age students; requires simple, engaging, and visual content
General	Broad public audience without specific background knowledge; mixed skill levels
Executive	Senior leaders or C-level stakeholders; values clarity, high-level summaries, and impact
Investor	Individuals or groups evaluating opportunities, such as startups or funding pitches
Government	Policy makers, civil servants, or public sector professionals
Workshop/training	Participants in hands-on or skill-building sessions
Conference audience	Mixed-level audience at events; may include peers, experts, or media

3.1.1. Article Generation

The dataset includes articles that are each associated with a main issue describing common presentation challenges, such as “text-heavy”, “readability”, “presentation skills”, or “data visualization”. The issues are linked to specific features that influence recommendations. Each article is defined by the following attributes:

- The article's name and summary;
- A predefined primary issue (e.g., "text-heavy");
- A list of related common presentation challenges (Table 2);
- A presentation type that the article is best suited for (Table 3);
- A relevant audience type (Table 4);
- A popularity score representing prior user engagement;
- The date when the article was submitted.

These attributes help the model align the article content with user needs and presentation contexts.

3.1.2. User Profile Generation

Users are randomly generated to reflect diverse roles and backgrounds. Each user profile is designed to simulate realistic behavior and preferences within the presentation domain. Key attributes include the following:

- User type (e.g., business professional, teacher, student, researcher, or manager);
- Geographic location;
- Presentation preferences, aligned with common presentation challenges (Table 2);
- Preferred presentation type (Table 3);
- Preferred audience type (Table 4).

These features enable the system to tailor recommendations based on personalized user contexts.

3.1.3. Presentation Generation

Presentations are generated for each user, containing multiple presentation issues. The system assigns timestamps and marks key presentation flaws, ensuring a structured dataset for recommendation learning. Each generated presentation includes the following:

- A user ID and unique presentation ID;
- A submission timestamp (randomized between 2005 and the present);
- A set of issues identified in the presentation (the same as the presentation challenges that are presented in Table 2).

Recent presentations are given higher priority in recommendations using a time decay function, ensuring that current problems influence suggestions more heavily than past issues.

3.1.4. Rating Generation

A ratings matrix is created based on multiple scoring factors to simulate how users evaluate recommendations:

- User preferences matching article attributes: higher scores for aligned articles;
- Presentation type match: boost for articles matching the user's preferred style;
- Audience type match: preference is given to content suitable for the user's target audience;
- Historical article popularity: articles with high past engagement receive a slight rating boost;
- Temporal weighting: recent presentation problems influence ratings more than older issues.

Each rating includes the user ID, article ID, rating score (a value in the range of $1 \div 5$), and the timestamp of the rating assignment. The final interaction dataset consists of time-aware recommendations, ensuring relevance to users' evolving needs.

3.2. Exploratory Analysis

The dataset consists of 25,000 entries with 66 columns capturing user interactions with articles, ratings, timestamps, and preferences related to presentation styles. Each user and article are uniquely identified with ratings ranging from two to five and a timestamp indicating when the interaction occurred. The dataset includes user preferences for various aspects of presentations, such as “readability”, “graphics”, “images”, “bullets”, “text size”, and “infographics”, all represented as binary indicators. Additionally, user types such as business professionals, teachers, students, researchers, managers, technical experts, and specialists are recorded. Presentation types, including formal, creative, student-oriented, technical, and branding-focused, are also categorized. The dataset further includes audience types, such as academic, business, kids, general, creative, and specialized, as well as location-based information. All columns contain complete data with no missing values. The summary statistics indicate an even distribution of users, and the articles had an average rating of approximately 3.09 (Figure 1). Figure 2 shows a heatmap of how user and article features are related. Figure 3 presents the average preferences of users, which illustrates what most users care about in their presentations.

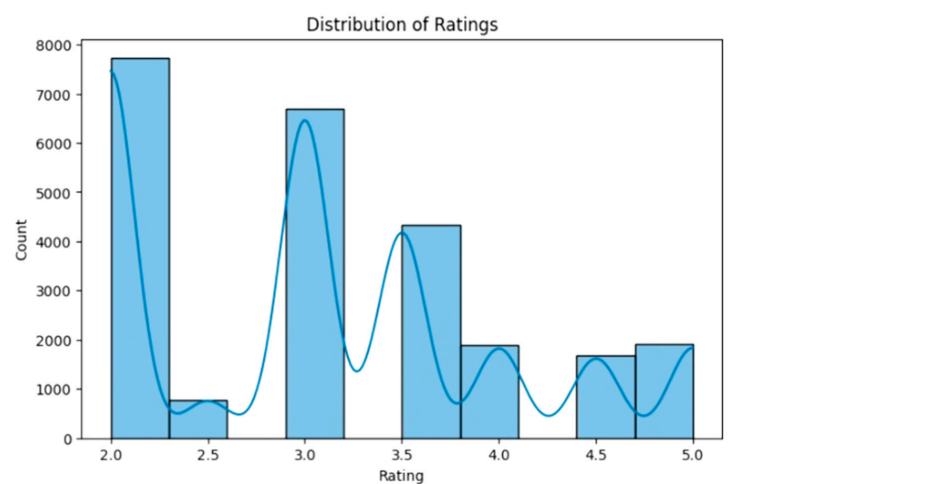


Figure 1. Distribution of ratings in the dataset.

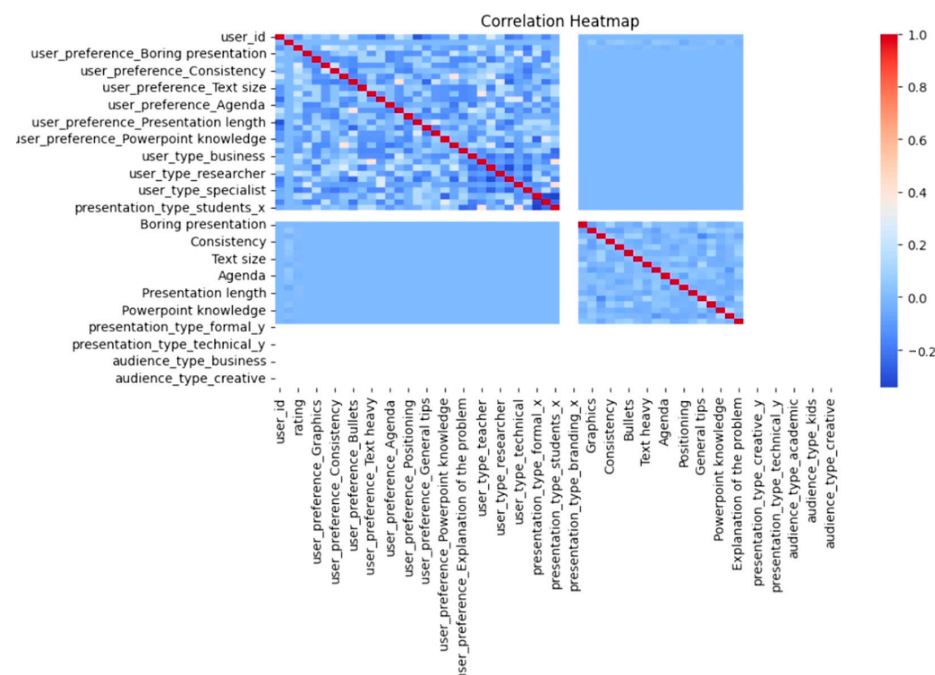


Figure 2. Correlation heatmap of user/article features.

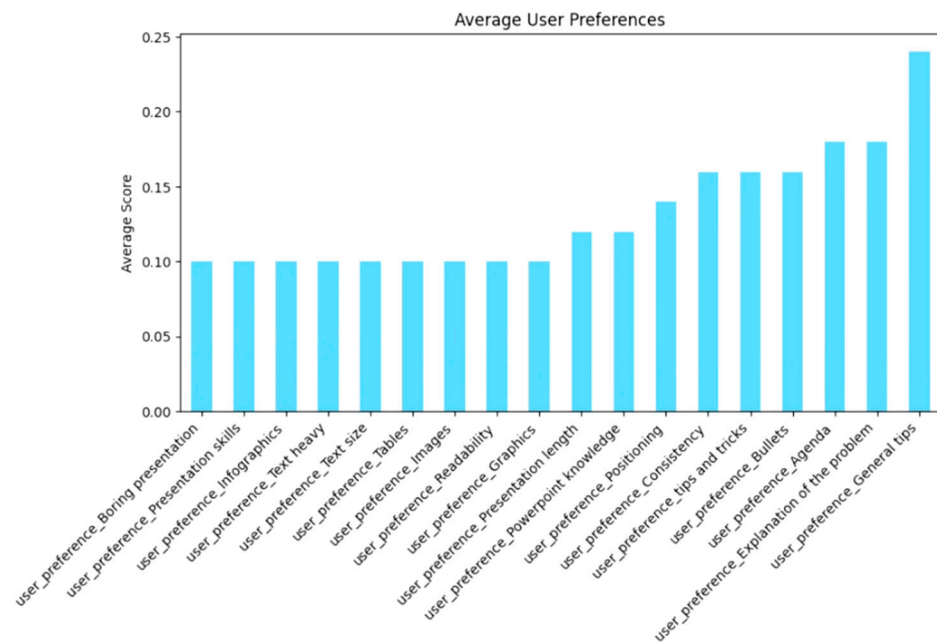


Figure 3. Average user preferences.

4. Recommender System Design

4.1. Recommender System's Role in the Presentation Advisor

The recommender system is the cornerstone of a presentation advisor application. Its primary role is to curate and present relevant content, such as articles, blogs, or YouTube videos, to users. This tailored content is designed to assist users in enhancing their presentation skills effectively.

The recommendations are generated based on several key factors:

1. **User profile:** This includes the user's personal preferences, expertise level, and previous interactions with the system. The system leverages this information to provide personalized and context-aware suggestions;
2. **Current presentation:** The content of the ongoing presentation is analyzed to identify its style, structure, and potential areas for improvement. This allows the system to recommend resources that align closely with the specific needs of the current work;
3. **Previous presentations:** Insights are drawn from past presentations, especially the problems or areas of improvement identified in those projects. This historical data ensure that recommendations address recurring issues and help the user build on their past experiences;
4. **User preferences and constraints:** The system takes into account user-defined preferences, such as the following:
 - the type of presentation (e.g., formal, educational, or persuasive);
 - the target audience (e.g., executives or students).

The system also takes into account several constraints:

- **Attention span:** Users who spend less time on the system receive fewer, more concise recommendations to align with their limited availability;
- **Location-based accessibility:** Certain articles or materials are filtered out based on the user's region, recognizing cultural sensitivity. For example, what works well in the United States might be inappropriate or even offensive in parts of Asia, so recommendations are curated to ensure cultural relevance and respect.

By synthesizing these inputs, the recommendation system is aimed at delivering highly targeted suggestions, facilitating the users to not only address immediate presentation

challenges but also to improve their overall presentation skills over time. This approach ensures that users receive practical, actionable, and contextually relevant guidance tailored to their unique needs and circumstances.

4.2. Recommendation System Architecture

The Presentation Advisor application is designed as a modular, service-based system that analyzes PowerPoint presentations and gives relevant personalized suggestions to improve content quality and visual consistency (Figure 4). The system includes a front-end user interface and a back-end composed of services that handle processing, analysis, and recommendation tasks. Users start using the Presentation Advisor by interacting with the User Interface (Login/Signup Module and Presentation Upload Interface). After logging in and uploading a file, the presentation goes to the Slide Preprocessor Module. The module is responsible for separating the presentation file into individual slides and preparing them for further analysis. This processing step allows us to standardize the input and supports faster processing by the next services.

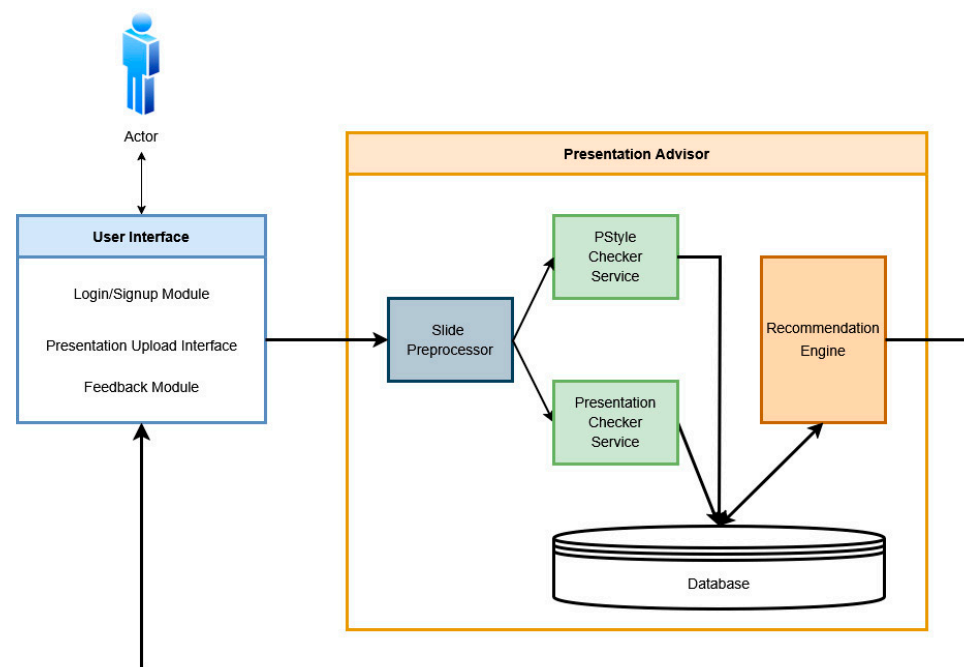


Figure 4. Recommender system architecture.

The slides prepared by the Slide Preprocessor Module are then sent to two different analysis services:

- **PStyle checker service:** Evaluates the coherence of the presentation slides using a custom-built AI model to assess the consistency of a presentation's visual style [53]. It compares the slides in pairs and identifies any slides that deviate from the overall style. If all slides are consistent, it returns a value of one; otherwise, the value returned is an index of the inconsistent slide(s) as an array. In the Presentation Advisor (PA), a score of zero indicates inconsistency, while one indicates consistency;
- **Presentation checker service:** This module identifies structural and content-related issues in the presentation, such as overcrowded slides, a poor layout, and the ineffective use of multimedia via multiple AI modules [54]. The detected issues are mapped to specific presentation problem categories (as described in Table 2) and are fed into the Presentation Advisor (PA) as an array of corresponding values.

The results from both services are saved in a centralized database. The main part of the system is the recommendation engine. It uses a hybrid recommendation model that

combines content-based filtering with learning from different types of data. The engine uses the results from the two analysis services as well as past user presentations' metadata and user profile information from the database. The system produces suggestions that are not only correct but also useful for learning and relevant to each user's situation.

4.3. Recommender Model Architecture

The recommendation model is designed as a custom hybrid solution utilizing a multi-tower neural network and utility-based scoring (Figure 5), which process user features, article attributes, temporal interactions, and problem sequence data through separate towers before combining them for feature fusion.

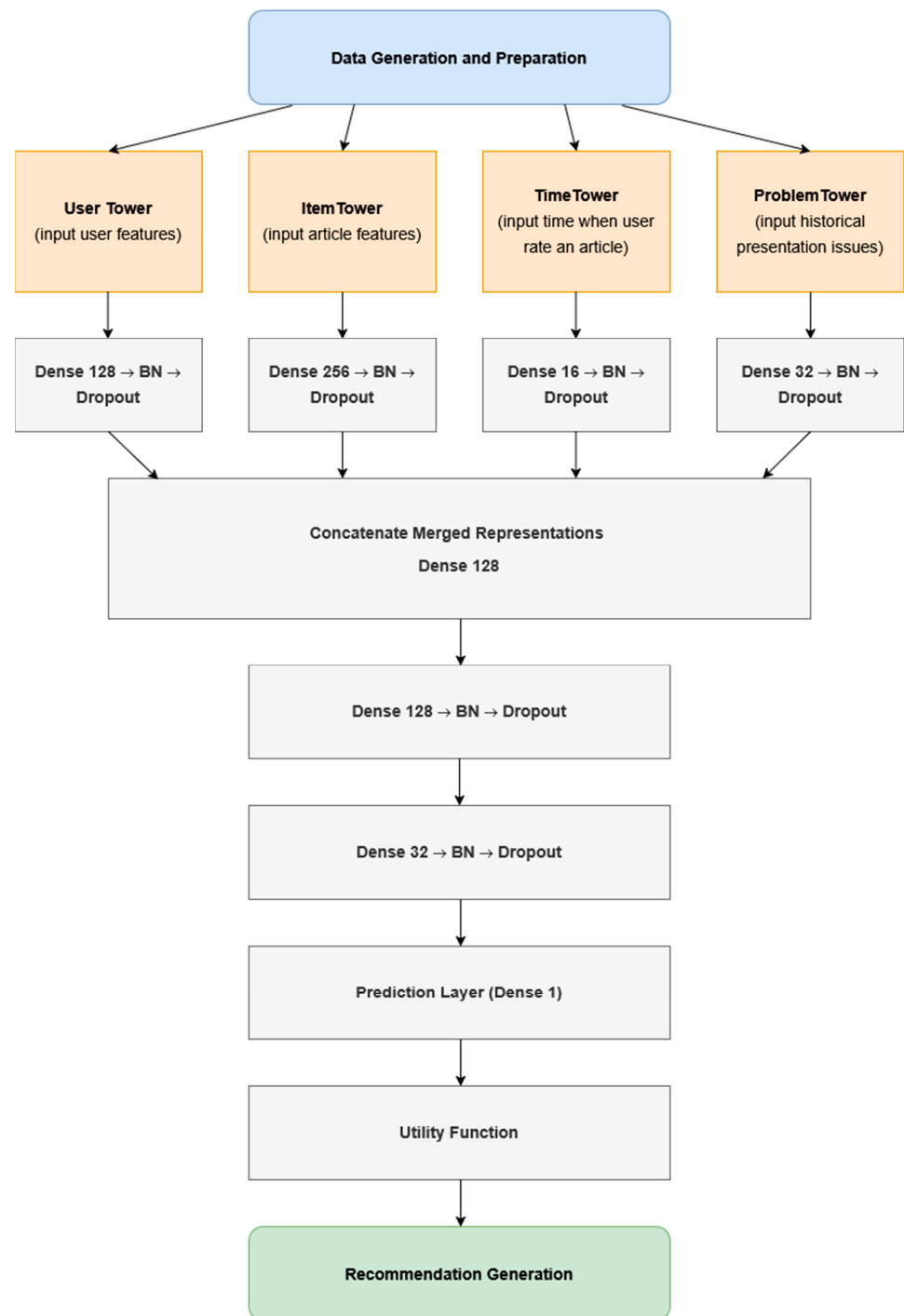


Figure 5. Recommender model architecture.

The suggested structure allows the model to capture multiple aspects of user behavior and content relevance, thus ultimately improving the recommendation quality. The system also incorporates an exponential decay weighting function to prioritize recent problem-solving history over older interactions, ensuring that recommendations are dynamically aligned with the user's evolving needs.

During the preprocessing stage, articles are filtered to eliminate cultural and geographical mismatches, thus ensuring contextually relevant recommendations while maintaining the constraints. To optimize the model architecture and hyperparameters, KerasTuner is used for automatic fine-tuning. Even though the users' features and the items/articles' features are the same size in the model architecture, the user and item tower have very different sizes. The difference can be explained based on the silhouette score evaluation of the dataset (Figure 6). The user data cluster more naturally (the silhouette score is 0.2019), meaning that users tend to form distinct well-defined groups based on their features. In contrast, the item data are more dispersed and heterogeneous with a lower silhouette score of 0.1051, indicating weaker clustering. The item space has a naturally greater amount of variety, leading to a wider spread of feature patterns. As a result, a user tower with a smaller number of neurons (128 in the suggested architecture) is sufficient to model the more consistent and clustered user behavior while a larger item tower is necessary to capture the broader diversity and complexity of presentations that require a greater representational capacity.

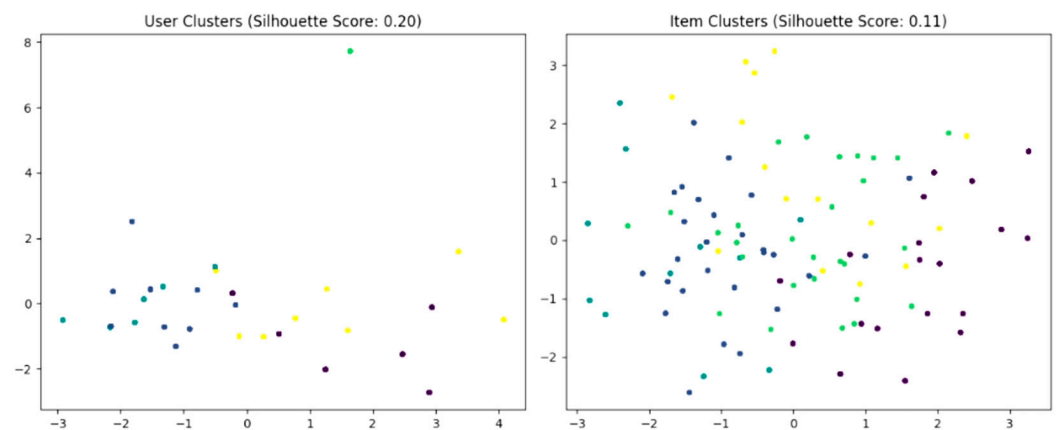


Figure 6. Silhouette score evaluation of user and item clusters.

The architecture also addresses cold-start issues by leveraging a feature-based approach. For new users, recommendations are generated using available attributes such as demographics and preferences. For new items (articles), the model relies solely on content features/metadata, making it adaptable to unseen articles without requiring prior interactions. Additionally, partial training is employed to update the model with new data while retaining learned knowledge, preventing overfitting and ensuring seamless content integration. Once article scores are predicted, a utility-based scoring mechanism is applied to refine recommendations further. This approach considers multiple contextual factors, including presentation style preferences, user engagement levels, and audience alignment, ensuring that recommendations are both personalized and impactful.

5. Methodology for Training the Recommender Model

5.1. Feature Extraction

To enhance recommendation accuracy, the system employs feature extraction and time-encoding techniques. User features are structured into a feature matrix incorporating binary user preferences such as “user_preference_text_heavy” along with one-hot

encoded user types like “user_type_teacher” and presentation preferences such as “presentation_type_formal_x”. Similarly, item features include binary attributes like “text heavy” and “graphics” as well as encoded presentation and audience types, ensuring effective content alignment.

Temporal features play a crucial role in capturing user engagement patterns. The dataset tracks the time since a user’s first submission in hours to understand the evolution of their engagement. Additionally, cyclic time encoding is applied by converting timestamps into sinusoidal representations using Hour_sin, Hour_cos, Day_sin, and Day_cos to reflect daily and weekly behavioral trends. This approach enables the system to learn both long-term user preferences and short-term fluctuations in engagement.

To maintain recommendation relevance over time, a problem sequence weighting mechanism is implemented using an exponential decay function. This function prioritizes recently occurring presentation challenges while reducing the influence of older issues. The padded problem sequence is transformed into a numerical array, ensuring it remains dynamic and adaptable to evolving user concerns. By incorporating these techniques, the recommendation system continuously refines personalization, making it responsive to users’ changing needs.

5.2. Model Fine-Tuning

Model fine-tuning is essential for developing an efficient recommender system as demonstrated in numerous studies [55–58]. The proposed recommender model is fine-tuned using KerasTuner, which automates the search for optimal hyperparameters to enhance accuracy and efficiency. The model follows a multi-tower neural architecture, in which user features, item features, temporal features, and problem sequence embeddings are processed separately before merging for final predictions. Each component undergoes hyperparameter optimization to ensure it captures meaningful and non-redundant feature interactions.

KerasTuner’s Hyperband algorithm is employed to efficiently explore a wide configuration space, dynamically adjusting key parameters such as the number of units per layer, dropout rates, embedding sizes, and the learning rate. The defined search space for this tuning process includes the following:

- The units per tower: {64, 128, 256};
- The embedding size: {16, 32, 64};
- The dropout rate: [0.1–0.5];
- The learning rate: [10^{-5} , 10^{-3}] (log-uniform distribution).

The search space is defined to dynamically select the number of units in the user, item, time, and problem sequence towers from 64, 128, or 256. Dropout rates vary in the range from 0.2 to 0.5 to regularize the network, while the size of user and item embeddings are fine-tuned between 16, 32, and 64 dimensions. Additionally, the merged representation is tuned to balance feature fusion effectively.

To ensure the reproducibility of training outcomes, the Mersenne Twister random number generator is explicitly initialized by the Python functions `np.random.seed(42)` and `tf.random.set_seed(42)`.

The tuner iteratively evaluates multiple model variants and selects the best-performing configuration based on validation loss. To further reduce overfitting, an early stopping mechanism halts training when no improvement is observed. Once the optimal parameters are identified, the final model is retrained using these values, thereby enhancing generalization and delivering a higher recommendation accuracy. This automated fine-tuning approach enables the system to adapt dynamically to diverse user and item profiles without manual intervention.

5.3. Model Training

In order to train and evaluate the suggested recommender model, the data generation, the model design, and the required tools for model training and evaluation are implemented using Python 3.8 and Tensorflow 2.5.0. The model is trained using the suggested architecture on the custom generated dataset. The dataset is split into training, validation, and test datasets (70%, 15%, and 15%); thus, the training dataset comprises 17,500 records and the validation dataset and the test dataset have 3750 records each. The model is trained for 200 epochs, utilizing five-fold cross-validation for hyper parameter tuning with a batch size of 32 and a learning rate set to 5.17×10^{-4} . Early stopping is also used as a regularization strategy to prevent overfitting. The parameters of the model are given in Table 5. The model is trained on the following experimental platform: Intel Core i7-8700K CPU@ 3.70 GHz, 64 GB RAM.

Table 5. Parameters of the recommender system model.

Layer Name	Type	Output Shape	Parameters	Connected To
user_feat_input	InputLayer	(None, 30)	0	-
item_feat_input	InputLayer	(None, 29)	0	-
problem_seq_input	InputLayer	(None, 12)	0	-
time_input	InputLayer	(None, 5)	0	-
dense_user	Dense	(None, 128)	3968	user_feat_input
dense_item	Dense	(None, 256)	7680	item_feat_input
dense_problem_seq	Dense	(None, 32)	416	problem_seq_input
dense_time	Dense	(None, 16)	96	time_input
bn_user	BatchNormalization	(None, 128)	512	dense_user
bn_item	BatchNormalization	(None, 256)	1024	dense_item
bn_time	BatchNormalization	(None, 16)	64	dense_time
bn_problem_seq	BatchNormalization	(None, 32)	128	dense_problem_seq
dropout_user	Dropout	(None, 128)	0	bn_user
dropout_item	Dropout	(None, 256)	0	bn_item
dropout_time	Dropout	(None, 16)	0	bn_time
dropout_problem_seq	Dropout	(None, 32)	0	bn_problem_seq
dense_user_embed	Dense	(None, 16)	2064	dropout_user
dense_item_embed	Dense	(None, 16)	4112	dropout_item
dense_problem_seq_proj	Dense	(None, 32)	1056	dropout_problem_seq
concat_embedding	Concatenate	(None, 80)	0	dense_user_embed, dense_item_embed, dropout_time, dense_problem_seq_proj
dense_combined	Dense	(None, 128)	10,368	concat_embedding
bn_combined	BatchNormalization	(None, 128)	512	dense_combined
dropout_combined	Dropout	(None, 128)	0	bn_combined
dense_hidden	Dense	(None, 32)	4128	dropout_combined
output	Dense	(None, 1)	33	dense_hidden

5.4. Cold-Start Problem

One of the major challenges in recommendation systems in general is the cold-start problem, in which the model struggles to generate meaningful recommendations for new users or new items. Prior research has proposed frequent pattern mining as an effective strategy for cold-start mitigation [59,60]. The suggested recommendation model presented in the article builds on this idea by leveraging user and item attributes to generate initial recommendations even with minimal interaction data [61,62]. The proposed strategy for mitigating the cold-start problem includes the following:

- Cold-start for users: Since the suggested recommender model is feature-based, it can leverage available user attributes, such as demographics, past interactions, and engagement history, even if the user has minimal interaction data. This allows the system to generate reasonable recommendations from the very beginning;
- Cold-start for items (articles): To mitigate this issue, the suggested model's item branch is only feature-based, thus ensuring that it can adapt to new articles effectively, without even requiring prior interactions with them. This feature-based approach allows for the seamless integration of new items while also preventing overfitting to older, more familiar content;
- Problem sequence tower (pre-trained): Even if a new user has limited history (to only the current presentation), the pre-training on problem sequences allows the model to generalize issue patterns, thus allowing it to make knowledge-based guesses justified on the basis of similar users.

For new users with no data, demographic and preference features from the user's onboarding are utilized as user type, preferred presentation type, and audience. To be able to receive recommendations, all users should have at least one presentation submitted. These are the two components required to match with relevant articles as they allow for the calculation of the matching score of each article and the user.

5.5. Model Retraining Strategy

As more new items become available, incremental fine-tuning is used to retrain the model only on newly collected data while keeping the previously learned weights. To achieve this, a lower learning rate is utilized for fine-tuning to preserve past knowledge. If the performance degrades, full retraining is applied [63]. This retraining strategy allows the computational overhead to be reduced by avoiding unnecessarily undergoing full retraining all the time and adapts quickly to new user interactions without losing older knowledge. The strategy is particularly effective since the new dataset is expected to be small and structurally similar to the existing data.

5.6. Utility-Based Scoring/Filtering

To enhance recommendation relevance, the proposed approach uses utility-based scoring, which adjusts the model's scores by considering contextual constraints and user-specific factors. The integrated utility-based scoring strategy balances personal preferences with broader content signals such as freshness, diversity, and fairness. By allowing the system to prioritize highly relevant content while also promoting serendipitous discovery and catalog coverage, the recommendation relevance improves, while also avoiding over-personalization.

At the core of the suggested utility-based scoring strategy is a weighted utility score calculated as follows:

$$\text{utility_score} = \text{predicted_rating} \times (1 + \alpha \times \text{audience_boost}) \times (1 + \beta \times \text{long_tail_boost}) \quad (1)$$

where:

- α is a tunable weight that controls the strength of the audience preference boost;
- β is a tunable weight for the long-tail boost that promotes less popular or newer items. A small β ensures diversity and freshness without over-prioritizing niche content;
- `predicted_rating` reflects the user's preference as predicted by the multi-tower model;
- `audience_boost` enhances content aligned with the user's preferred audience type, helping personalize results without making them too narrow;
- `long_tail_boost` enhances the visibility of lesser-known or recently added content by combining two factors: content freshness and inverse popularity. This boost is integrated into the overall recommendation score using carefully tuned weights, ensuring that diverse and underexposed items gain exposure without overshadowing highly relevant results. The goal is to improve discovery, diversity, and catalog fairness while preserving strong personalization and content relevance.

The long-tail boost is calculated as follows:

$$\text{long_tail_boost} = \gamma \times \text{freshness_score} + (1 - \gamma) \times \text{inverse_popularity} \quad (2)$$

where:

- `freshness_score` is normalized based on how recently the article is submitted;
- `inverse_popularity` rewards rare or under-engaged items;
- γ controls the balance between freshness and rarity.

To ensure recommendation fairness and diversity and keep personalization from dominating the ranking, ensuring that the boosts enhance but do not overpower the base prediction, the weight values used are as follows:

$$\alpha = 0.2, \beta = 0.3, \gamma = 0.5 \quad (3)$$

As real user data are collected in the future system deployment, the parameter tuning will be based on offline evaluation (precision, recall, novelty, and diversity) and online evaluation (click-through rate, time spent, and user feedback).

The sub-steps of the suggested approach for utility-based scoring include the following:

- Presentation type matching: Articles matching the user's preferred presentation style are given higher priority;
- Engagement time adjustment: The number of recommendations is adjusted based on the user's time spent in the system, reducing suggestions for less engaged users;
- Audience type matching: Each article is assigned a binary boost depending on whether its tagged audience matches the user profile;
- Enhanced utility calculation: All factors are combined to compute a final utility score for each recommendation, ensuring high levels of relevance and impact;
- Long-tail boosting: A combined metric derived from both submission freshness and rarity (inverse popularity), enhancing the visibility of under-represented content.

Combining the suggested approach for utility-based scoring with the recommender engine results is aimed to provide the best recommendations tailored to the user's needs. The final utility score is calculated per item and used to rank the results.

6. Challenges and Solutions

6.1. Data Sparsity

One of the primary challenges in building a hybrid recommendation model is data sparsity, which occurs when user-item interactions are limited, making it difficult for the model to learn effective patterns. To address this issue, data augmentation and transfer learning are applied. Data augmentation techniques, such as generating synthetic interac-

tions based on existing patterns, are used to enrich the dataset. Transfer learning is also employed by pre-training the model on a similar larger dataset before fine-tuning it on the target dataset. These approaches allow for the improvement of the model's ability to generalize from sparse data.

6.2. Context Understanding

Ensuring that recommendations are contextually relevant is crucial for user satisfaction. This challenge is tackled by incorporating additional contextual features, such as time of interaction and user location, into the model. By integrating this contextual information, the model can provide more relevant recommendations that align with the user's current situation. The effectiveness of contextual modeling in recommender systems has been demonstrated in previous research [60], so the suggested model builds upon this by integrating time-dependent and audience-related features.

6.3. Technical Challenges

Several technical challenges are encountered during the development of the model. One significant complication is managing the model's complexity due to the integration of multiple input types—sequential and non-sequential data. To overcome this challenge, the suggested architecture is optimized to ensure efficient processing. Furthermore, parallel computing techniques are used to speed up the model training. Another issue is model overfitting, especially given the limited data used. To mitigate model overfitting, several regularization techniques are employed, like dropout and early stopping, alongside hyperparameter tuning to find the optimal model configuration. These solutions collectively enhance the model's performance and robustness.

6.3.1. Incremental Learning

The challenge of continuous learning from new data without retraining the entire model is addressed by using partial training, which allows the system to incrementally learn from new interactions, ensuring that updates are efficient and the model remains current without exhaustive retraining.

6.3.2. Time-Dependent Data

Handling time-dependent data is crucial for delivering timely recommendations. To address this challenge, a dedicated tower is incorporated in the model architecture for temporal data that capture a user rating an article. Additionally, pre-training is applied to the presentation sequence using time-decayed weighting to emphasize recent issues, ensuring the model prioritizes currently relevant problems.

6.3.3. Long-Tail Recommendations

A common issue in RSs is the tendency to recommend popular items, neglecting less popular (long-tail) items, which reduces the level of exposure to new, niche, or lesser-known items. To address this issue, long-tail boosting is added to the suggested recommender engine.

7. Evaluation and Experimental Results

7.1. Evaluated Models

7.1.1. Content-Based Recommendations and Collaborative Filtering

The baseline model for the performance comparison uses the most basic implementation of a content-based RS and a collaborative filtering RS [3,64].

7.1.2. Autoencoders

Autoencoders excel in dimensionality reduction and feature extraction, as well as in learning compact representations of users and items [64]. They are particularly useful for tasks like collaborative filtering, data compression, addressing the cold-start problem, and denoising recommendations. Autoencoders perform poorly with sparse data such as the data of the suggested recommender system for the Presentation Advisor application.

7.1.3. Reinforcement Learning

A deep Q-network (DQN) [65] is used for the suggested recommendation model presented in the article. A DQN agent is a strong choice because it enables the model to learn from user interactions and adjust its recommendations over time. DQNs use deep learning to approximate the optimal Q-values, allowing the agent to effectively explore a large action space and balance exploration with the exploitation of the most rewarding articles. This makes DQNs particularly suited for environments with a dynamic evolving set of interactions such as personalized recommendations based on user preferences.

The recommendation model acts as an agent while the dataset of users and articles serves as an environment. The agent's task is to recommend articles to users, observe their reactions, and adjust future recommendations based on those reactions. Each recommendation scenario is represented as a state which includes both user-specific and article-specific features. The action space consists of the articles available, from which the agent selects one to recommend at each step. The system assigns rewards based on user engagement, in which the reward value matches the true rating of the user's interaction with the article.

The gym environment is implemented through the "PresentationRecommenderEnv" class, which simulates the interactions between users and articles. The DQN agent is a neural network model used to predict the expected reward for each possible article recommendation. The model consists of three fully connected layers: 256 neurons, 128 neurons, and an output layer that provides Q-values for each article. As part of the prediction process, the model takes both user features and all item features producing a prediction for each article.

A key challenge in recommendation systems is the cold-start problem, in which new users or new items have limited historical interaction data. Traditional methods like collaborative filtering struggle with this but reinforcement learning (RL) models like DQNs can handle it well by balancing exploration and exploitation.

Example scenarios in which a DQN is used are as follows:

- **New User:** When a new user joins, the RL model initially explores by recommending a diverse set of articles. Based on user interactions (e.g., clicks), it assigns rewards and learns preferences. For example, if the user engages in technology-related articles, the model remembers this interaction and prioritizes similar content in future recommendations;
- **New Item:** When a new article is added, the RL model includes it in random recommendations to evaluate user interest. If users engage positively, the model assigns higher Q-values to the article, increasing its likelihood of being recommended to relevant users. Conversely, if engagement is low, the article gradually becomes less recommended.

The suggested approach allows the model to dynamically adapt to both new users and new items, improving the recommendation process without relying solely on historical data. As for the issue of handling the cold-start problem, a pre-training strategy is also considered. Since RL continuously updates its policy based on new interactions, retraining is different from traditional machine learning models. However, there are specific scenarios when retraining from scratch or fine-tuning the model is necessary. The model should

be retrained when there are significant changes in user behavior, new users, new items, or performance degradation. Fine-tuning (continuing training on new data) works well for small updates while full retraining from scratch is needed for major shifts. The model can also adapt continuously by periodically updating with recent interactions. Scheduled retraining should be triggered when performance metrics (like the MSE) degrade beyond a threshold. As the model inherently follows the core principles of temporal difference (TD) learning and Q-learning, frequent retraining is not expected. Since the Q-values are updated incrementally after each interaction using the Bellman equation, the model continuously learns and adapts over time. This allows it to refine its recommendations dynamically based on new experiences, reducing the need for full retraining.

7.1.4. Hybrid Model with Custom Pre-Trained Embeddings

This model implements a hybrid recommendation architecture that combines collaborative filtering and content-based strategies using a multi-input deep learning framework. The core idea is to integrate both semantic features (e.g., slide content and user role) and behavioral patterns (e.g., past interactions and sequence of problems) in a meaningful and scalable way.

The key components of the suggested hybrid model with custom pre-trained embeddings are follows:

- Custom pre-trained user–item embeddings: User and item features (e.g., profile data, metadata, and behavioral attributes) are encoded using embeddings that are pre-trained on domain-specific interactions. These embeddings are learned via a separate unsupervised model;
- Hand-crafted presentation embeddings: The presentations themselves significantly influence the ratings, and their dynamic nature makes it difficult to capture them accurately with learned embeddings. Users frequently upload new presentations, often at every session, making it computationally expensive and impractical to learn embeddings for them. Therefore, hand-crafted embeddings for presentations are used instead;
- Multi-tower architecture: The model uses dual-tower (or multi-tower) neural architecture with the user–item embeddings as one tower and presentations embeddings as the other. The output of the model is the predicted relevance score. The model is trained with the mean squared error (MSE) as the optimization goal;
- Utility-based scoring: For downstream ranking or evaluation, a post-processing layer adjusts the predicted scores using utility-based criteria, balancing relevance with novelty and diversity.

7.2. Performance Comparison

Utility scoring is applied after the recommendation, and this functionality is crucial as it allows to rank and filter recommendations based on multiple factors beyond just predicted ratings. It helps to balance different objectives such as relevance (predicted rating), popularity, recency, and non-mandatory user preferences. The performance results of the compared RS models based on prediction accuracy metrics (using the MAE, MSE, and RMSE) and model training time are given in Table 6. Tables 7 and 8 show the top 10 recommendations before and after using the suggested utility scoring, respectively. As can be seen from the results, the hybrid model provides better results than the other evaluated models in terms of its lower error values and has a good training time, especially when pre-trained embeddings are added. After utility scoring is applied, the recommendations are re-ordered to better match the users' needs by also taking into account how recent the ratings are.

Table 6. Evaluation of RS models.

Model	MAE	MSE	RMSE	Timing
CBF+CF	1.13	1.96	1.40	0.29 s
Autoencoders	3.05	10.46	3.23	~1 h
RL (DQN)	3.22	12.22	3.49	~15 min
Hybrid model	0.11	0.033	0.18	~25 min
Hybrid model + custom pre-trained embeddings	0.49	0.36	0.60	~1 h

Table 7. Top 10 recommendations before utility scoring.

Rank	Article ID	True Rating	Predicted Score	Days Since User Rated
1	9	4.5	4.77493	3413.88
2	8	5.0	4.717092	3587.75
3	6	4.5	4.642761	1771.29
4	35	4.5	4.627602	2197.96
5	17	4.5	4.605495	3002.75
6	3	4.5	4.59516	7124.75
7	41	4.5	4.577928	560.04
8	26	4.5	4.568518	7069.38
9	46	4.5	4.562939	6663.63
10	25	5.0	4.555358	7126.96

Table 8. Final recommendations after utility scoring.

Rank	Article ID	True Rating	Predicted Score	Days Since User Rated	Utility Score
1	9	4.5	4.77493	3413.88	5.48542
2	8	5.0	4.717092	3587.75	5.407775
3	6	4.5	4.642761	1771.29	5.174015
4	17	4.5	4.605495	3002.75	5.149448
5	25	5.0	4.555358	7126.96	5.100156
6	35	4.5	4.627602	2197.96	5.088142
7	31	4.5	4.423069	5007.63	5.083353
8	3	4.5	4.59516	7124.75	5.005856
9	26	4.5	4.568518	7069.38	4.926949
10	63	4.5	4.27723	3257.38	4.887227

As can be seen from the experimental evaluation of the RS model, the best performance metrics are achieved for the hybrid model. As expected, the CBF+CF autoencoders and RL model do not perform very well because some of them are too simple and others are too complicated for the dataset used. The second-best model is the hybrid model with custom pre-trained embeddings. The performance degradation in this case compared to the pure hybrid model is due to the model not being well aligned with the nature of the data. The decision to use custom embeddings is based on the observation that most of

the features are numerical and do not benefit from pre-trained word embeddings such as GloVe that are designed primarily for textual data. In the suggested hybrid model with custom pre-trained embeddings, only embeddings for users, recommendations, and ratings are learned as these components exhibit relatively stable patterns. Due to the dynamic nature of the presentations and their influence on the ratings, hand-crafted embeddings for presentations are used. This approach, while more efficient, results in a reduced accuracy compared to the multi-tower architecture.

In addition, paired statistical significance tests are performed across the five-fold cross-validation results of our two best models to assess whether the observed improvements in the MAE are robust. The suggested model is compared to the baseline model with pre-trained embeddings using the following:

- A paired *t*-test: $t = -4.8362$, $p = 0.0084$;
- The Wilcoxon signed-rank test: $W = 0.0000$, $p = 0.0625$.

The statistical significance tests indicate that the improvement in validation MAE is statistically significant ($p < 0.01$), supporting the effectiveness of the proposed model.

A summary of the recommendation list changes after applying utility scoring is given in Table 9. The table compares the rankings of the recommended items before and after utility scoring, thus indicating how the inclusion of time-related factors—such as the recency of user interactions—affects the final order. The aim of utility scoring is to improve recommendation relevance by not only considering predicted ratings but also reflecting how recent and meaningful those interactions are.

Table 9. Comparison of the recommendations ranking before and after utility scoring.

Before (Ranked by Predicted Rating)	After (Ranked by Utility Score)
The top recommendations are ranked only by model prediction score	Recommendations are ranked by utility score, balancing a blend of predicted score and time-aware utility
Items with high rating but old interactions (e.g., articles 25, 26, and 3) are placed at the top	Newer articles (e.g., articles 6, 8, and 9) are boosted, given priority due to recency
Older articles with high rating but low levels of recent user interaction are over-recommended	Time decay effect: older but still relevant articles are considered but pushed lower
No time sensitivity; stale but high-scoring items remain on top	Recency of interactions reorders results for better engagement

Additional metrics of the recommendation model as described in [3] are evaluated using the test dataset (Table 10):

- Loss (0.2122): Loss is a general measure of model error during training, typically representing the difference between predicted and actual outcomes. Lower values indicate better performance in terms of minimizing prediction errors;
- Precision (1): Precision evaluates the proportion of recommended items in the top-k list that are relevant to the user. A higher precision value indicates that the model often recommends items that the user is likely to find valuable;
- Novelty (0.45): Novelty measures how “unknown” or uncommon the recommended items are to the user. A higher novelty score indicates that the system is suggesting less popular or previously unseen items, contributing to increased user interest and engagement;
- Serendipity (0.42): Serendipity assesses how surprising yet relevant the recommendations are. A high serendipity score shows that the system can recommend unexpected items that still align with the user’s preferences, providing delightful discoveries;

- Diversity (0.49): Diversity measures how varied the recommended items are. A higher diversity score suggests that the system provides a broader range of items, reducing the likelihood of recommending highly similar or redundant items;
- Item Coverage (0.15): Item coverage reflects the proportion of the total item catalog that is recommended. A low score indicates that the model focuses on a small subset of items, potentially limiting the variety of recommendations and excluding some relevant items from the suggestions.

Table 10. Beyond-accuracy evaluation metrics of the recommendation model.

Metric	Value	Metric	Value
Loss	0.2122	Serendipity	0.42
Precision	1	Diversity	0.49
Novelty	0.45	Item Coverage	0.15

8. System Deployment and Maintenance Strategy

To ensure consistent performance in real-world conditions, the recommender system uses a modular deployment and maintenance strategy that includes three independently deployed components:

- PStyle checker: Assesses the visual and structural coherence of presentation slides;
- Presentation checker: Identifies common issues in slides using classification models;
- Recommendation engine: Suggests relevant articles based on user behavior and context.

Each model is managed and monitored separately, allowing for targeted updates and easy scaling.

8.1. Deployment Infrastructure

Models are deployed using TensorFlow Serving [66] within Docker containers, ensuring consistent and scalable hosting. Each model version is stored in a versioned directory and served via API. This setup supports A/B testing, batch predictions, and fast roll-backs if needed. The deployment is cloud-ready and compatible with platforms like GCP Vertex AI, AWS SageMaker, and Kubernetes, allowing for auto-scaling, monitoring, and CI/CD integration.

8.2. Real-Time Inference Pipeline

When the recommender system is used in real time, user inputs (preferences, context, and engagement) are sent to the appropriate model through an REST API. The model returns predictions, which are saved in a database. These predictions are later used as input with the recommendation engine that generates suggestions. The final output is refined using a utility-based scoring module that considers relevance, novelty, long-tail content, and freshness. This real-time inference pipeline ensures fast, personalized, and context-aware recommendations.

8.3. CI/CD and Monitoring

A CI/CD pipeline handles model validation, versioning, and rollout [67]. Real-time monitoring (via Prometheus and Grafana) tracks metrics like accuracy, precision, recall, and user feedback to maintain high model performance.

8.4. Periodic Retraining via Replay-Based Learning

To maintain model relevance over time without retraining from scratch, the recommender system adopts a periodic retraining strategy using replay-based continual learn-

ing [68]. Unlike online or incremental learning, this approach involves retraining the model at regular intervals using a curated set of newly labeled data reflecting real-world usage patterns and a representative subset of historical data to preserve previously learned knowledge. The workflow for the periodic retraining using via replay-based learning is as follows:

1. Data preparation and model training: The model is retrained on a combined dataset of new and sampled old data for a small number of epochs. EarlyStopping and ModelCheckpoint callbacks are used to avoid overfitting;
2. Validation: The updated model is validated on both a static test set to ensure historical performance is preserved and a new test set to assess performance on recent data;
3. Deployment: If the validation criteria are met, the model is versioned and deployed through the CI/CD pipeline.

The key benefits of using the suggested workflow for periodic retraining via replay-based learning are as follows:

- Maintains historical knowledge while adapting to new trends;
- Efficiently uses computational resources;
- Reduces risk of catastrophic forgetting;
- Enables targeted improvements in underperforming areas.

By combining containerized deployment, modular architecture, replay-based periodic retraining, and continuous monitoring, the presented recommender system ensures high availability, adaptability, and long-term performance. Each of the modules—the PStyle checker, presentation checker, and recommendation engine, are maintained independently to support scalability, responsiveness, and resilience in production environments.

9. Conclusions

The presented study describes the design and development of a custom recommendation system model tailored for a presentation advisor application, designed to help users enhance their presentations by delivering relevant and personalized article recommendations. The recommendation system is built on a custom dataset, allowing for a more targeted approach to recommendation generation. Various state-of-the-art methodologies for recommender systems are explored, including content-based filtering, collaborative filtering, autoencoders, and reinforcement learning. A multi-tower neural network architecture is proposed as the optimal model for the considered recommendation task. The suggested architecture provides balance between prediction accuracy, computational efficiency, and processing speed. To further improve recommendation quality, the system incorporates feature engineering and problem sequence weighting, enabling it to adapt to users' evolving needs. Utility-based scoring is also suggested to refine the final recommendations by aligning them with user preferences, engagement levels, and contextual constraints, thus ensuring a more personalized experience. The proposed recommendation system effectively addresses the cold-start problem by leveraging both feature-based representations and historical interactions. Additionally, the integration of exponential decay weighting ensures that recommendations remain relevant by prioritizing recent presentation challenges.

Even though the presented framework serves as a strong starting point, the taxonomy of presentation challenges will be expanded in the future deployment of the system. Real-world user interaction will guide the refinement and enrichment of the categories used to more comprehensively reflect the complexities of actual presentations.

While synthetic data generation enabled rapid prototyping, we recognize its limitations with respect to external validity. In the absence of publicly available labeled datasets

for presentation recommendation, this approach provided a viable starting point. However, as external validation is essential, we plan to conduct evaluations using real-world presentation data and user feedback in upcoming studies.

As part of the future work, an assessment of the system effectiveness and pedagogical impact is planned based on a longitudinal user study with at least 50 participants from four major user groups: students, teachers, business professionals, and researchers. The study will use evaluation metrics, such as pre/post skill assessment, the engagement time per session, the retention of changes (how many recommendations are accepted), and self-reported confidence and satisfaction levels using the system. The aim is also to integrate LLMs to enhance the semantic understanding of presentations and generate richer and more adaptive feedback as LLMs offer great potential to support personalized educational and recommendation systems and can significantly augment the contextual reasoning capabilities of the proposed model.

The presented work establishes a context-aware, adaptive, and personalized recommendation engine, laying a strong foundation for improving user experience in presentation refinement. It also outlines a realistic path forward for system deployment, external validation, future challenges and expansion, and enhanced capability through LLM integration.

Supplementary Materials: The following supporting information can be downloaded at: <https://www.mdpi.com/article/10.3390/electronics14132528/s1>, dataset used for the presented research.

Author Contributions: Conceptualization, M.V.-T. and M.L.; methodology, M.V.-T.; software, M.V.-T.; validation, M.V.-T.; formal analysis, M.V.-T.; investigation, M.V.-T. and M.L.; resources, M.V.-T.; data curation, M.V.-T.; writing—original draft preparation, M.V.-T. and M.L.; writing—review and editing, M.V.-T. and M.L.; visualization, M.V.-T.; supervision, M.L.; project administration, M.L.; funding acquisition, M.L. All authors have read and agreed to the published version of the manuscript.

Funding: The research work presented in the paper is funded by European Union—NextGenerationEU via the National Recovery and Resilience Plan of the Republic of Bulgaria under project BG-RRP-2.004-0005 “Improving the research capacity and quality to achieve international recognition and reSilience of TU-Sofia (IDEAS)”.

Data Availability Statement: Data are contained within the article or Supplementary Material.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Neuxpower: Slidewise PowerPoint Add-In. Available online: <https://neuxpower.com/slidewise-powerpoint-add-in> (accessed on 27 May 2025).
2. PitchVantage. Available online: <https://pitchvantage.com> (accessed on 27 May 2025).
3. Ricci, F.; Rokach, L.; Shapira, B. Recommender systems: Techniques, applications, and challenges. In *Recommender Systems Handbook*, 3rd ed.; Ricci, F., Rokach, L., Shapira, B., Eds.; Springer: New York, NY, USA, 2022; pp. 1–35.
4. Raza, S.; Rahman, M.; Kamawal, S.; Toroghi, A.; Raval, A.; Navah, F.; Kazemeini, A. A comprehensive review of recommender systems: Transitioning from theory to practice. *arXiv* **2024**, arXiv:2407.13699.
5. Fayyaz, Z.; Ebrahimian, M.; Nawara, D.; Ibrahim, A.; Kashef, R. Recommendation systems: Algorithms, challenges, metrics, and business opportunities. *Appl. Sci.* **2020**, *10*, 7748. [\[CrossRef\]](#)
6. Saifudin, I.; Widiyaningtyas, T. Systematic literature review on recommender system: Approach, problem, evaluation techniques, datasets. *IEEE Access* **2024**, *12*, 19827–19847. [\[CrossRef\]](#)
7. Alfaifi, Y.H. Recommender systems applications: Data sources, features, and challenges. *Information* **2024**, *15*, 660. [\[CrossRef\]](#)
8. Jeong, S.-Y.; Kim, Y.-K. State-of-the-art survey on deep learning-based recommender systems for e-learning. *Appl. Sci.* **2022**, *12*, 11996.
9. Zhang, Y.; Chen, X. Explainable recommendation: A survey and new perspectives. *arXiv* **2020**, arXiv:1804.11192. [\[CrossRef\]](#)
10. Ko, H.; Lee, S.; Park, Y.; Choi, A. A Survey of recommendation systems: Recommendation models, techniques, and application fields. *Electronics* **2022**, *11*, 141. [\[CrossRef\]](#)

11. Aljunid, M.; Manjaiah, D.; Hooshmand, M.; Ali, W.; Shetty, A.; Alzoubah, S. A collaborative filtering recommender systems: Survey. *Neurocomputing* **2025**, *617*, 128718. [\[CrossRef\]](#)
12. Kim, T.-Y.; Ko, H.; Kim, S.-H.; Kim, H.-D. Modeling of recommendation system based on emotional information and collaborative filtering. *Sensors* **2021**, *21*, 1997. [\[CrossRef\]](#)
13. Beheshti, A.; Yakhchi, S.; Mousaeirad, S.; Ghafari, S.M.; Goluguri, S.R.; Edrisi, M.A. Towards cognitive recommender systems. *Algorithms* **2020**, *13*, 176. [\[CrossRef\]](#)
14. Chicaiza, J.; Valdiviezo-Diaz, P. A comprehensive survey of knowledge graph-based recommender systems: Technologies, development, and contributions. *Information* **2021**, *12*, 232. [\[CrossRef\]](#)
15. Troussas, C.; Krouska, A.; Tselenti, P.; Kardaras, D.K.; Barbounaki, S. Enhancing Personalized educational content recommendation through cosine similarity-based knowledge graphs and contextual signals. *Information* **2023**, *14*, 505. [\[CrossRef\]](#)
16. Uta, M.; Felfernig, A.; Le, V.; Tran, T.; Garber, D.; Lubos, S.; Burgstaller, T. Knowledge-based recommender systems: Overview and research directions. *Front. Big Data* **2024**, *7*, 1304439. [\[CrossRef\]](#) [\[PubMed\]](#)
17. Chaudhari, A.; Hitham Seddig, A.; Sarlan, A.; Raut, R. A hybrid recommendation system: A review. *IEEE Access* **2024**, *12*, 157107–157126. [\[CrossRef\]](#)
18. Mouhiha, M.; Oualhaj, O.; Mabrouk, A. Combining collaborative filtering and content based filtering for recommendation systems. In Proceedings of the 2024 11th International Conference on Wireless Networks and Mobile Communications (WINCOM), Leeds, UK, 23–25 July 2024.
19. Singh, K.; Dhawan, S.; Bali, N. An Ensemble learning hybrid recommendation system using content-based, collaborative filtering, supervised learning and boosting algorithms. *Autom. Control. Comput. Sci.* **2024**, *58*, 491–505. [\[CrossRef\]](#)
20. Shahbazi, Z.; Byun, Y.C. Toward social media content recommendation integrated with data science and machine learning approach for e-learners. *Symmetry* **2020**, *12*, 1798. [\[CrossRef\]](#)
21. Al-Nafjan, A.; Alrashoudi, N.; Alrasheed, H. Recommendation System algorithms on location-based social networks: Comparative study. *Information* **2022**, *13*, 188. [\[CrossRef\]](#)
22. Bakhshizadeh, M. Supporting Knowledge workers through personal information assistance with context-aware recommender systems. In Proceedings of the 18th ACM Conference on Recommender Systems (RecSys'24), Bari, Italy, 14–18 October 2024; pp. 1296–1301.
23. Afzal, I.; Yilmazel, B.; Kaleli, C. An Approach for multi-context-aware multi-criteria recommender systems based on deep learning. *IEEE Access* **2024**, *12*, 99936–99948. [\[CrossRef\]](#)
24. Shrivastava, R.; Sisodia, D.; Nagwani, N. Utility optimization-based multi-stakeholder personalized recommendation system. *Data Technol. Appl.* **2022**, *56*, 782–805. [\[CrossRef\]](#)
25. Tansuchat, R.; Kosheleva, O. How to make recommendation systems fair: An adequate utility-based approach. *Asian J. Econ. Bank.* **2022**, *6*, 308–313. [\[CrossRef\]](#)
26. Gheewala, S.; Xu, S.; Yeom, S. In-depth survey: Deep learning in recommender systems—exploring prediction and ranking models, datasets, feature analysis, and emerging trends. *Neural Comput. Appl.* **2025**, *37*, 10875–10947. [\[CrossRef\]](#)
27. Devika, P.; Milton, A. Book recommendation using sentiment analysis and ensembling hybrid deep learning models. *Knowl. Inf. Syst.* **2025**, *67*, 1131–1168. [\[CrossRef\]](#)
28. Tran, H.; Chen, T.; Hung, N.; Huang, Z.; Cui, L.; Yin, H. A thorough performance benchmarking on lightweight embedding-based recommender systems. *ACM Trans. Inf. Syst.* **2025**, *43*, 1–32. [\[CrossRef\]](#)
29. Gomez-Urbe, C.A.; Hunt, N. The Netflix recommender system: Algorithms, business value, and innovation. *ACM Trans. Manag. Inf. Syst.* **2015**, *6*, 1–19. [\[CrossRef\]](#)
30. Steck, H.; Baltrunas, L.; Elahi, E.; Liang, D.; Raimond, Y.; Basilico, J. Deep learning for recommender systems: A Netflix case study. *AI Magazine* **2021**, *42*, 7–18. [\[CrossRef\]](#)
31. Nagrecha, K.; Liu, L.; Delgado, P.; Padmanabhan, P. InTune: Reinforcement learning-based data pipeline optimization for deep recommendation models. In Proceedings of the 17th ACM Conference on Recommender Systems (RecSys'23), Singapore, 18–22 September 2023; pp. 430–442.
32. Barwal, D.; Joshi, S.; Obaid, A.J.; Abdulbaqi, A.S.; Al-Barzinji, S.M.; Alkhayyat, A.; Hachem, S.K.; Muthmainnah. The impact of netflix recommendation engine on customer experience. *AIP Conf. Proc.* **2023**, *2736*, 060005.
33. Smith, B.; Linden, G. Two decades of recommender systems at Amazon.com. *IEEE Internet Comput.* **2017**, *21*, 12–18. [\[CrossRef\]](#)
34. Hardesty, L. The history of Amazon's recommendation algorithm. *Amaz. Sci.* **2019**. Available online: <https://www.amazon.science/the-history-of-amazons-recommendation-algorithm> (accessed on 18 June 2025).
35. Zhao, J.; Wang, L.; Xiang, D.; Johanson, B. Collaborative denoising auto-encoders for top-N recommender systems. In Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR'19), Paris, France, 21–25 July 2019.
36. Zhao, Z.; Fan, W.; Li, J.; Liu, Y.; Mei, X.; Wang, Y. Recommender systems in the era of large language models (LLMs). *IEEE Trans. Knowl. Data Eng.* **2024**, *36*, 6889–6907. [\[CrossRef\]](#)

37. Lin, J.; Dai, X.; Xi, Y.; Liu, W.; Chen, B.; Zhang, H.; Liu, Y.; Wu, C.; Li, X.; Zhu, C.; et al. How can recommender systems benefit from large language models: A survey. *ACM Trans. Inf. Syst.* **2025**, *43*, 1–47. [\[CrossRef\]](#)
38. Pellegrini, R.; Zhao, W.; Murray, I. Don't recommend the obvious: Estimate probability ratios. In Proceedings of the 16th ACM Conference on Recommender Systems (RecSys'22), Seattle, WA, USA, 18–23 September 2022; pp. 188–197.
39. Zhang, Y.; Ding, H.; Shui, Z.; Ma, Y.; Zou, J.; Deoras, A.; Wang, H. Language models as recommender systems: Evaluations and limitations. In Proceedings of the NeurIPS 2021 Workshop on I (Still) Can't Believe It's Not Better, Virtual, 13 December 2021.
40. Yu, T.; Ma, Y.; Deoras, A. Achieving diversity and relevancy in zero-shot recommender systems for human evaluations. In Proceedings of the NeurIPS 2022 Workshop on Human in the Loop Learning, New Orleans, LA, USA, 2 December 2022.
41. Lessa, L.F.; Brandao, W.C. Filtering Graduate Courses based on LinkedIn Profiles. In Proceedings of the WebMedia 2018, Salvador, Brazil, 16–19 October 2018.
42. Urdaneta-Ponte, M.C.; Oleagordia-Ruiz, I.; Méndez-Zorrilla, A. Using linkedin endorsements to reinforce an ontology and machine learning-based recommender system to improve professional skills. *Electronics* **2022**, *11*, 1190. [\[CrossRef\]](#)
43. He, X.; Liao, L.; Zhang, H.; Nie, L.; Hu, X.; Chua, T.S. Neural collaborative filtering. In Proceedings of the 26th International Conference on World Wide Web, Perth, Australia, 3–7 April 2017; pp. 173–182.
44. Zhang, X.; Zhou, Y.; Ma, Y.; Chen, B.C.; Zhang, L.; Agarwal, D. GLMix: Generalized linear mixed models for large-scale response prediction. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 13–17 August 2016; pp. 363–372.
45. Davidson, J.; Liebald, B.; Liu, J.; Nandy, P.; Van Vleet, T.; Gargi, U.; Gupta, S.; He, Y.; Lambert, M.; Livingston, B.; et al. The YouTube video recommendation system. In Proceedings of the 4th ACM Conference on Recommender Systems (RecSys'10), Barcelona, Spain, 26–30 September 2010; pp. 293–296.
46. Ma, J.; Zhao, Z.; Yi, X.; Chen, J.; Hong, L.; Chi, E.H. Modeling task relationships in multi-task learning with multi-gate mixture-of-experts. In Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, London, UK, 19–23 August 2018; pp. 1930–1939.
47. Covington, P.; Adams, J.; Sargin, E. Deep neural networks for YouTube recommendations. In Proceedings of the 10th ACM Conference on Recommender Systems (RecSys'16), Boston, MA, USA, 15–19 September 2016; pp. 191–198.
48. Vlahova, M.; Lazarova, M. Collecting a custom database for image classification in recommender systems. In Proceedings of the 10th International Scientific Conference on Computer Science (COMSCI), Sofia, Bulgaria, 30 May–2 June 2022.
49. Green, E. The basics of slide design. In *Healthy Presentations*; Springer: Cham, Switzerland, 2021; pp. 37–62.
50. Duarte, N. *Slide:ology—The Art and Science of Creating Great Presentations*; O'Reilly: Sebastopol, CA, USA, 2008.
51. Gallo, G. *Talk Like TED: The 9 Public Speaking Secrets of the World's Top Minds*; St. Martin's Press: New York, NY, USA, 2014.
52. Jambor, H.; Bornhäuser, M. Ten simple rules for designing graphical abstracts. *PLoS Comput. Biol.* **2024**, *20*, e1011789. [\[CrossRef\]](#)
53. Vlahova-Takova, M.; Lazarova, M. Dual-branch convolutional neural network for image comparison in presentation style coherence. *Eng. Technol. Appl. Sci. Res.* **2025**, *15*, 21719–21727. [\[CrossRef\]](#)
54. Vlahova-Takova, M.; Lazarova, M. CNN based multi-label image classification for presentation recommender system. *Int. J. Inf. Technol. Secur.* **2024**, *16*, 73–84. [\[CrossRef\]](#)
55. Bernardini, L.; Bono, F.M.; Collina, A. Drive-by damage detection based on the use of CWT and sparse autoencoder applied to steel truss railway bridge. *Adv. Mech. Eng.* **2025**, *17*. [\[CrossRef\]](#)
56. Qian, F.; Cui, Y.; Xu, M.; Chen, H.; Chen, W.; Xu, Q.; Wu, C.; Yan, Y.; Zhao, S. IFM: Integrating and fine-tuning adversarial examples of recommendation system under multiple models to enhance their transferability. *Knowl.-Based Syst.* **2025**, *11*, 113111. [\[CrossRef\]](#)
57. Tiep, N.; Jeong, H.-Y.; Kim, K.-D.; Xuan Mung, N.; Dao, N.-N.; Tran, H.-N.; Hoang, V.-K.; Ngoc Anh, N.; Vu, M. A New Hyperparameter Tuning Framework for Regression Tasks in Deep Neural Network: Combined-Sampling Algorithm to Search the Optimized Hyperparameters. *Mathematics* **2024**, *12*, 3892. [\[CrossRef\]](#)
58. Moscati, M.; Deldjoo, Y.; Carparelli, G.; Schedl, M. Multiobjective hyperparameter optimization of recommender systems. In Proceedings of the 3rd Workshop Perspectives on the Evaluation of Recommender Systems (PERSPECTIVES 2023), Singapore, 19 September 2023.
59. Panteli, A.; Boutsinas, B. Addressing the cold-start problem in recommender systems based on frequent patterns. *Algorithms* **2023**, *16*, 182. [\[CrossRef\]](#)
60. Jeong, S.-Y.; Kim, Y.-K. Deep Learning-Based Context-Aware Recommender System Considering Contextual Features. *Appl. Sci.* **2021**, *12*, 45. [\[CrossRef\]](#)
61. Lv, X.; Fang, K.; Liu, T. Content-aware few-shot meta-learning for cold-start recommendations using cross-modal attention. *Sensors* **2024**, *24*, 5510. [\[CrossRef\]](#)
62. Luo, Y.; Jiang, Y.; Jiang, Y.; Chen, G.; Wang, J.; Bian, K.; Li, P.; Zhang, Q. Online item cold-start recommendation with popularity-aware meta-learning. In Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, V.1 (KDD'25), Toronto, Canada, 3–7 August 2025; Association for Computing Machinery: New York, NY, USA; pp. 927–937.

63. Ding, S.; Feng, F.; He, X.; Liao, Y.; Shi, J.; Zhang, Y. Causal incremental graph convolution for recommender system retraining. *IEEE Trans. Neural Netw. Learn. Syst.* **2024**, *35*, 4718–4728. [[CrossRef](#)] [[PubMed](#)]
64. Zhang, S.; Yao, L.; Sun, A.; Tay, Y. Deep learning-based recommender system: A survey and new perspectives. *ACM Comput. Surv.* **2020**, *52*, 1–38. [[CrossRef](#)]
65. Mnih, V.; Badia, A.P.; Mirza, M.; Graves, A.; Lillicrap, T.; Harley, T.; Silver, D.; Kavukcuoglu, K. Asynchronous methods for deep reinforcement learning. In Proceedings of the 33rd International Conference on Machine Learning, Proceedings of Machine Learning Research 2016, New York City, NY, USA, 19–24 June 2016; Volume 48, pp. 1928–1937.
66. TensorFlow Serving. Available online: <https://github.com/tensorflow/serving> (accessed on 18 June 2025).
67. Parmar, T. Implementing CI/CD in Data Engineering: Streamlining Data Pipelines for Reliable and Scalable Solutions. *Int. J. Innov. Res. Eng. Multidiscip. Phys. Sci.* **2025**, *13*. [[CrossRef](#)]
68. Rensing, F.; Lwakatare, L.E.; Nurminen, J.K. Exploring the application of replay-based continuous learning in a machine learning pipeline. In Proceedings of the Workshops of the EDBT/ICDT 2025 Joint Conference, Barcelona, Spain, 25–28 March 2025; Boehm, M., Daudjee, K., Eds.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.